

Data Replication in a Single-Leader Environment

MongoDB Replica Set + Python Control Node

Mert Karahan Doğukan Topçu

CENG465 — Principles of Data-Intensive Systems

May 5, 2026

Project Overview

Goal: demonstrate single-leader replication on two physical macOS machines — no containers, real LAN.

- MongoDB rs0 replica set (PRIMARY + SECONDARY)
- Schema with version, log-index, and operation-id tracking
- Python/Flask control node & live dashboard
- Write concern toggle: `w=majority` vs `w=1`
- Background reconciler for async catch-up detection

Core question

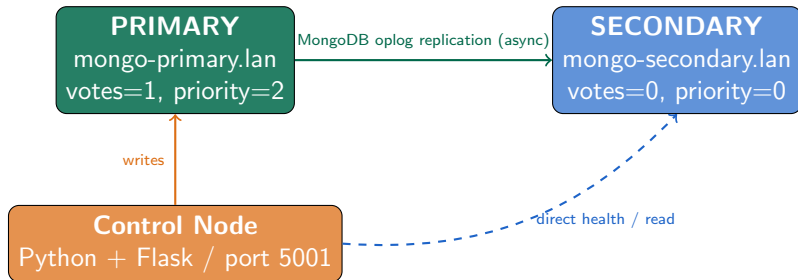
What happens when the follower goes offline?

- `w=majority`: reject the write
- `w=1`: accept the write, queue it
- How do we **observe** catch-up?

Machines

Primary	192.168.88.146
Secondary	192.168.88.105

Deployment Architecture



Why non-voting secondary?

With two voting members, disconnecting secondary removes majority — primary steps down and even $w=1$ cannot write. $votes=0$ keeps primary writable during outages.

Consequence

MongoDB alone satisfies $w=majority$ from the primary. We add an application-level guard: reject before any primary mutation if secondary cannot be pinged.

Replica Set Configuration

Initial issue

- Two voting members: primary + secondary
- Disconnecting secondary removed majority
- MongoDB reported ReplicaSetNoPrimary
- Even w=1 could not write — no writable primary

```
cfg = rs.conf()
cfg.members[1].priority = 0
cfg.members[1].votes = 0
rs.reconfig(cfg)
```

Solution: non-voting read replica

- Primary: votes=1, priority=2
- Secondary: votes=0, priority=0

Primary stays writable during follower outages.

Schema Design

items collection

```
{
  key,
  value,
  version,           -- increments on each write
  leader_term,
  last_log_index,
  last_operation_id,
  last_updated,
  deleted            -- soft delete
}
```

operation_logs collection

```
{
  operation_type,    -- insert / update / delete
  log_index,
  leader_write_time,
  follower_visible_time,
  replication_delay_ms,
  version_before,
  version_after,
  status,            -- pending_follower
                    -- visible_on_follower
                    -- timeout
  write_concern      -- majority / 1
}
```

Raft-inspired metadata

leader_term, log_index, version, and operation_id make write ordering and follower visibility fully observable.

Write Concern Modes

Mode	Secondary Up	Secondary Down
w=majority	Write succeeds; replication delay is measured and logged	Write rejected before primary mutation — no data change
w=1	Write succeeds immediately; log is reconciled in background	Write succeeds on primary; log stays pending_follower until catch-up

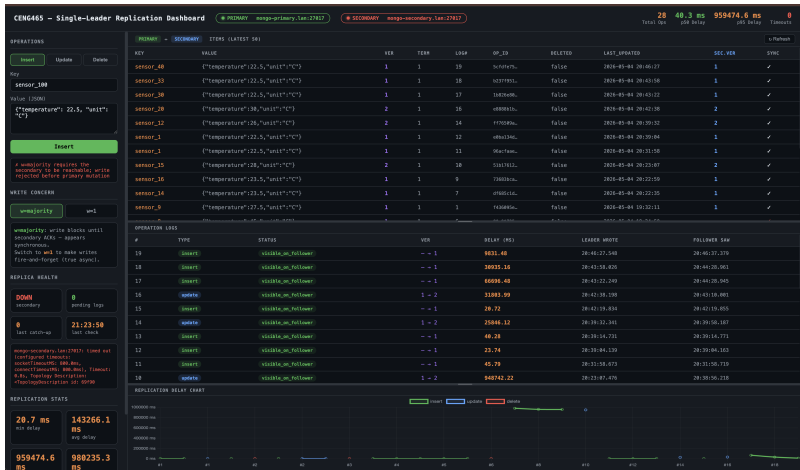
w=majority — Consistency-first

Reject the write rather than risk divergence. No partial state.

w=1 — Availability-first

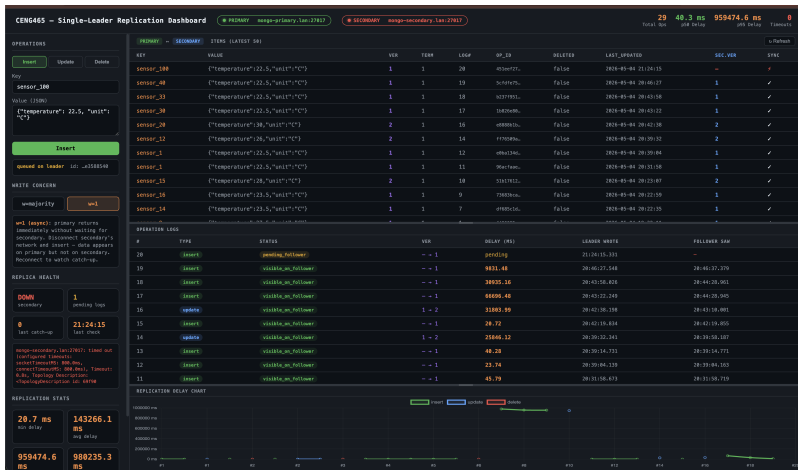
Accept the write; guarantee eventual delivery via oplog.

Demo Step 1 — w=majority Write Rejected



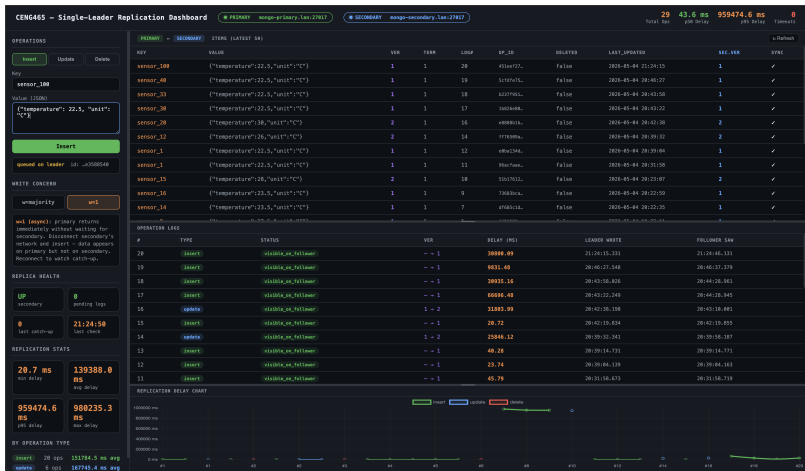
Secondary offline. Insert **rejected before primary mutation** — no new item, no pending log created.

Demo Step 2 — w=1 Write Succeeds



Switched to w=1. Insert completes instantly: sensor_100 on primary (ver=1), sec.ver = ---. Log: pending_follower.

Demo Step 3 — Secondary Reconnects & Catches Up



Oplog replayed; reconciler detects version match. Log: visible_on_follower; sec.ver matches; sync ✓ restored.

What we proved

- Replication is **asynchronous by default**
- Write concern controls the **durability–availability tradeoff**
- $w=\text{majority}$: consistency over availability
- $w=1$: availability with eventual catch-up via oplog
- **Observability** (version, delay, log-index) is essential

Implementation highlights

- Application-level write guard for $w=\text{majority}$
- Background reconciler (1 s cadence)
- `directConnection=true` for honest secondary health
- Version + log-index fallbacks for catch-up detection

Key insight

Single-leader replication gives strong write ordering but does **not** guarantee read-after-write consistency on followers.